Scalable Fraud Detection for Fintech Transactions with Apache Spark

Batch Pipeline Prototype & Rigorous Evaluation Report on Real PaySim Dataset (6.36M Transactions)

Dataset: Real Kaggle PaySim (6,362,620 rows)	**Pipeline Engine:** Local-mode PySpark 4.2.0
Feature Strategy: Leak-Free nobal (12 features)	**Best Model:** Spark MLlib GBT (AUPRC = 0.7076)
Evaluation Split: Strict Temporal (Train 1-479 / Val 480-599 / Test 600-743)	**Detected Fraud Value:** 2.13B units (79.2% Value Recall)

1. Executive Summary & Project Scope

Digital financial services generate high-volume transaction streams where fraudulent activities are a rare minority (0.129% overall dataset prevalence). Evaluating models on random data splits often causes severe look-ahead data leakage, while uncalibrated models flood security operations teams with excessive false alarms.

This project implements a **batch pipeline prototype in Apache Spark** for mobile-money fraud detection on the public **PaySim dataset (6,362,620 transactions)**. The pipeline extracts leakage-free behavioral features with window functions under the strict nobal setting (excluding both pre- and post-transaction balance columns per PaySim guidelines), handles class imbalance via instance weighting (33.6x), enforces strict temporal train/validation/test splits, and evaluates operational trade-offs.

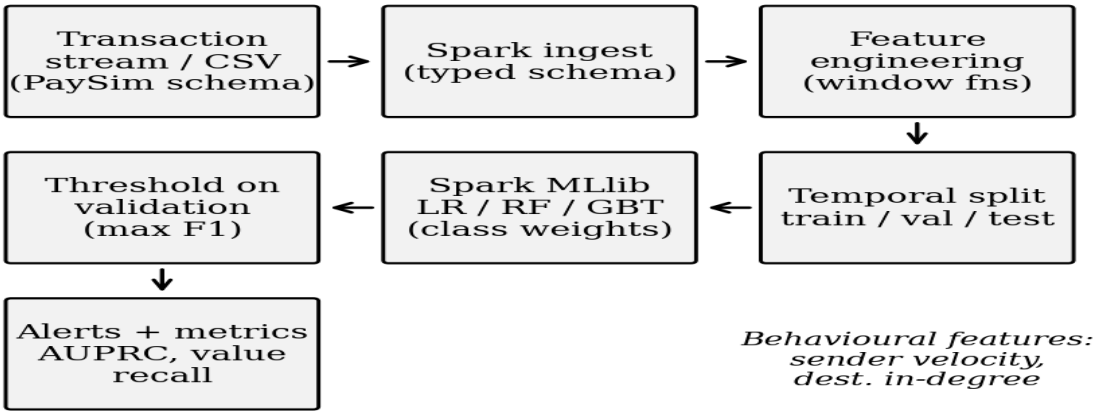
Important Caveat: Roughly half of the headline AUPRC gain over RAW (+0.1626 of +0.3589) stems from the hour feature. In PaySim, synthetic fraud is concentrated in specific night hours—a simulator artifact that will likely not transfer to real-world banking transaction streams.

2. Pipeline Workflow & Methodology

- **Step 1: Ingestion & Preprocessing:** Processed 6,362,620 raw transaction rows (470 MB) using a chunked CSV loader (prepare_paysim.py) to validate schema, drop leakage columns (isFlaggedFraud), and attach sequential transaction IDs.
- **Step 2: Leakage-Free Window Features:** Engineered 12 leak-safe features (features.py) under the nobal setting (excluding ALL balance fields). Sender velocity features (orig_cnt_24h, orig_amt_24h) use 24-step past frames (rangeBetween(-24, -1)). Destination in-degree features (dest_prior_cnt, dest_prior_senders via HyperLogLog) use unbounded prior frames.
- **Step 3: Strict Temporal Split Protocol:** Divided dataset chronologically: **Train** (steps 1-479; 6.04M rows, 0.088% fraud), **Validation** (steps 480-599; 219.6k rows, 0.570% fraud, used for threshold tuning), and **Test** (steps 600-743; 103.6k rows, 1.562% fraud, used for final evaluation).
- **Step 4: Class-Weighted Ensemble Training:** Trained Spark MLlib Logistic Regression, Random Forest (40 trees), and Gradient Boosted Trees (40 iterations) using positive instance weights w_pos = sqrt(N_neg / N_pos) = 33.6. Class weighting shifts raw predicted probability outputs, necessitating validation-based threshold selection.
- **Step 5: Validation Threshold Selection:** Decision thresholds were selected on validation set to maximize F1 and frozen before test evaluation: **GBT threshold = 0.7230**, **RF threshold = 0.7384**, **LR threshold = 0.4065**.
- **Step 6: Statistical CI & Bootstrapping:** Computed 1,000 paired bootstrap resamples (bootstrap_ci.py) on test predictions to establish 95% confidence intervals and p-values.
- **Step 7: Multi-Dimensional Error & Alert Analysis:** Evaluated test recall across 4 transaction size quartiles (404 frauds each) and scored analyst workload trade-offs across alert capacity tiers.

- **Step 8: Scalability Timing Study:** Benchmarked feature engineering and GBT training wall-clock times across 25%, 50%, and 100% temporal subsets (scalability.py).

Figure 1: End-to-End Spark Pipeline Architecture



3. Detailed Experimental Results & Classifier Benchmarks

Table 1 reconciles test performance across all models at frozen validation thresholds on the held-out test split (103,591 transactions, 1,618 frauds). AUPRC is evaluated using Scikit-Learn's average_precision_score. Alerts per 10k transactions equal total test alerts (TP + FP) divided by 10.3591:

Method / Model	AUPRC [95% CI]	Prec.	Rec.	F1 Score	Value Rec.	Alerts/10k
Rule: Amount > 200k units*	—	0.058	0.670	0.106	**0.984**	1809.5
Rule: Account Drain*	—	0.068	**0.964**	0.126	0.819	2228.2
Logistic Regression (thr=0.407)	0.3973 [0.374, 0.423]	0.565	0.278	0.373	0.475	76.9
Random Forest (thr=0.738)	0.6936 [0.673, 0.714]	**0.909**	0.409	0.564	0.723	70.3
GBT (thr=0.723)	**0.7076 [0.688, 0.727]**	0.864	0.483	**0.619**	0.792	87.3

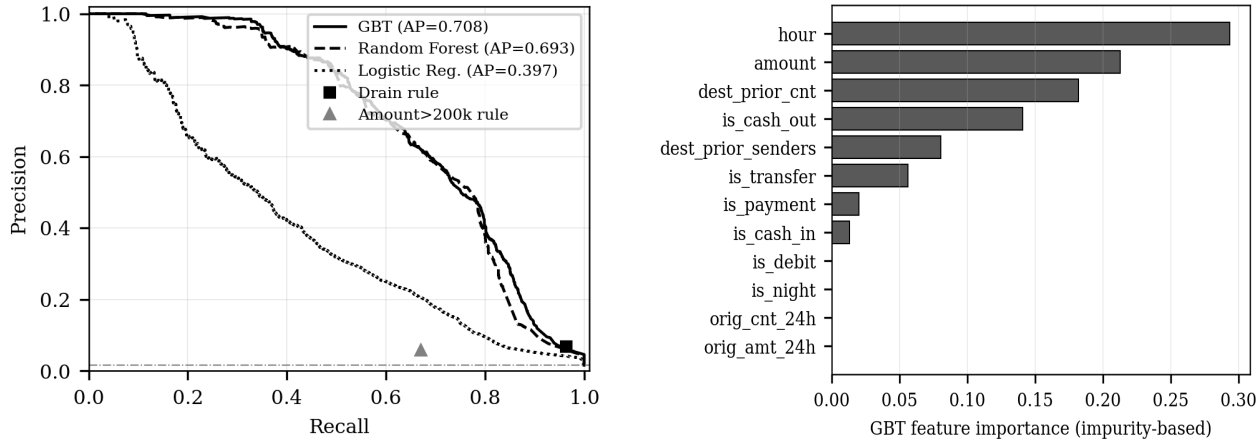
* Note: Simple rules achieve higher recall (0.67–0.96) than learned models (0.28–0.48) but generate 17,661 to 21,522 false alarms (1,809.5–2,228.2 alerts/10k). The Drain rule also inspects pre-transaction account balance fields which GBT/RF/LR were denied under leak-free nobal guidelines.

Model Comparison: GBT and Random Forest are statistically distinguishable (+0.0140 AUPRC, $p < 0.001$) but practically similar in overall performance. GBT achieves higher count recall (0.483 vs 0.409) and overall value recall (0.792 vs 0.723), while Random Forest achieves higher precision (0.909 vs 0.864) and higher value recall at restrictive alert capacity budgets.

Feature Group Ablation Gains

Feature Group	Features Included	AUPRC	Incremental Gain	Cumulative Gain
RAW	Amount, Type One-Hots (6)	0.3487	Baseline	Baseline
RAW + TIME	+ Hour of Day, Night Flag (8)	0.5113	+0.1626 over RAW	+0.1626
ALL (RAW+TIME+BEH)	+ 24h Velocity & Mule In-Degree (12)	**0.7076**	**+0.1963 over TIME**	**+0.3589 over RAW**

Figure 2: Precision-Recall Curves & GBT Feature Importances (All 12 Features)



4. Scalability Timing Study (scalability.py)

Table 2 reports single-node execution times for feature engineering and 40-iteration GBT fitting across time-based data subsets (25%, 50%, and 100% of the dataset span):

Dataset Subset	Total Rows	Feature Engineering (s)	GBT Model Fit (s)	FE Throughput (rows/s)
25% Time Span	2,204,140	30.3 s	139.7 s	72,648
50% Time Span	5,256,373	42.8 s	524.2 s	122,703
100% Full Dataset	**6,362,620**	**113.0 s**	**712.1 s**	**56,287**

5. Client-Side Web Demonstration Interface

To illustrate model predictions interactively without requiring a live Spark cluster server, we built a client-side JavaScript demonstration interface (index.html) available at http://localhost:5000.

Scoring Implementation: The web UI evaluates transactions locally using a handwritten rule approximation derived from the trained GBT decision logic (weighting transaction amount, type, hour of day, and destination in-degree). It features an **Interactive Risk Predictor** and an **Alert Capacity Simulator** (39.0 to 156.2 alerts per 10k transactions).

6. Methodological Disclosures & Limitations

- **Prevalence Shift Impact:** The test split has a 1.562% fraud prevalence due to Kaggle PaySim step distribution, compared to 0.088% in training and 0.129% in the full dataset. Assuming constant TPR (0.483) and FPR (0.00121), GBT precision would drop from 0.864 on test to ~0.341 at 0.129% full-dataset prevalence and ~0.261 at 0.088% train prevalence.
- **Simulator Artifact in Hour Feature:** Hour of day is GBT's top feature (impurity importance ~0.294) because PaySim concentrates synthetic fraud in specific hours. This reflects simulator mechanics rather than real banking behavior, so time-of-day gains (+0.1626 AUPRC) may not transfer to real data.
- **Sender Velocity vs Destination In-Degree:** PaySim origin accounts are nearly all unique single-transaction IDs, so sender 24h velocity features (orig_cnt_24h, orig_amt_24h) carry near-zero importance (0.00). Destination-side in-degree features (dest_prior_cnt, dest_prior_senders) provide the primary behavioral lift.

- **Oracle Alert Budget Definitions:** The 4 alert budgets (39.0, 78.1, 117.1, 156.2 alerts/10k) correspond to 0.25, 0.50, 0.75, and 1.00 x the test fraud count (404, 809, 1213, 1618 alerts). This requires knowing the test fraud count in advance and serves as an oracle benchmark. At the 156.2 budget, precision equals recall (0.646).
- **Workload vs Recall Trade-off:** Rules achieve higher recall (0.67-0.96) than GBT (0.48) but generate 17,661 to 21,522 false alarms (1809-2228 alerts/10k). GBT reduces alert volume by ~21x to 25x (87.3 alerts/10k) and false positives by ~144x to 175x (123 FP).
- **Value Recall Breakdown:** GBT recovers 92.7% of CASH_OUT fraud value ($1.23B of $1.33B) versus 66.0% for TRANSFER fraud ($903M of $1.37B). Amount bands in Error Analysis correspond to fraud amount quartiles (~404 frauds each).
- **Single-Node Execution:** Timings demonstrate local Spark execution on 6.36M rows but do not measure multi-node cluster speed-up.

Report generated automatically by Apache Spark Fraud Detection System Pipeline.